

Fast Track — Mock Exam 2 (Solutions)

ShopSphere scenario · full worked solutions

Prof. Dr. Christoph Weisser

HSBI

Summer Semester 2026

How to use these solutions

- These slides give a **fully worked answer** to every question of Mock Exam 2 (the ShopSphere paper), one question per slide.
- For **multiple-choice** questions the correct option is stated, followed by *why each distractor is wrong*.
- **Code** answers are runnable with the usual imports (`pandas as pd`, `numpy as np`, `scipy.stats`, `sklearn`, `bs4`); often more than one correct form exists — the marker rewards any correct idiom.
- Work each question yourself first, then check here. The mark for each item matches the paper.
- Total: **100 marks**, 5 parts of 20.

Outline

- 1 Part A — Python foundations
- 2 Part B — Data, pandas, viz & statistics
- 3 Part C — ML, evaluation & feature engineering
- 4 Part D — AI engineering
- 5 Part E — Applied & production

Correct: B) -3.

- `int()` on a float **truncates toward zero** — it drops the fractional part, it does *not* round. So `int(-3.9) → -3` and `int(3.9) → 3`.

Why the others are wrong:

- A) -4 would be `math.floor(-3.9)` (round toward $-\infty$) or `round(-3.9)` — not what `int()` does.
- C) 3 confuses sign; truncation keeps the sign.
- D) No error — `int()` of a float is perfectly valid.

Correct: B) ['mug', 'pen'].

- The default `cart=[]` is created **once, at definition time**, and *shared* across all calls that don't pass their own list. The first call appends "mug"; the same list survives, so the second call appends "pen" to it.

Fix (the None-sentinel idiom):

```
def add_to_cart(item, cart=None):  
    if cart is None:  
        cart = []  
    cart.append(item)  
    return cart
```

Distractors: A/C assume a fresh list per call (the trap); D ignores the append.

```
total = 1234.5
return_rate = 0.0873
print(f"\u20ac{total:,.2f}") # -> the euro sign then 1,234.50
print(f"{return_rate:.1%}") # -> 8.7%
```

Key points:

- `:,.2f` = thousands separator *and* 2 decimals $\Rightarrow$ 1,234.50.
- `:.1%` multiplies by 100 and appends % with 1 decimal $\Rightarrow$ 8.7% (0.0873 $\rightarrow$ 8.73 $\rightarrow$ rounded 8.7).
- 1 mark for the currency/thousands format, 1 for the percent format, 1 for exact rounding/output.

```
from collections import Counter

categories = ["mug", "pen", "mug", "lamp", "mug", "pen"]
counts = Counter(categories) # Counter({'mug': 3, 'pen': 2, 'lamp': 1})

top = max(counts, key=counts.get) # 'mug'
print(counts)
print(top)
```

Notes:

- Counter(iterable) tallies occurrences in one line.
- max(counts, key=counts.get) returns the key with the largest count; counts.most_common(1)[0][0] is an equally valid alternative.
- A manual d[k] = d.get(k, 0) + 1 loop also earns full marks.

Bug: inside the loop the code *overwrites* `total` with each price (`total = p`) instead of *adding* to it, so it returns only the last price.

```
def cart_total(prices):  
    total = 0  
    for p in prices:  
        total += p # was: total = p  
    return total
```

- The classic `=` vs `+=` accumulator mistake.
- Equivalent Pythonic one-liner: `return sum(prices)`.
- Marks: 2 for identifying the overwrite, 2 for the correct `+=` fix.


```
class ShoppingCart:
    TAX_RATE = 0.19 # class attribute, shared

    def __init__(self):
        self.items = [] # fresh list per instance (no mutable default)

    def add_item(self, name, price, qty=1):
        self.items.append({"name": name, "price": price, "qty": qty})

    def subtotal(self):
        return sum(it["price"] * it["qty"] for it in self.items)

    def total(self):
        return self.subtotal() * (1 + self.TAX_RATE)
```

Marks: class attr [1]; items in `__init__` not as a default [1]; `add_item` [1]; `subtotal` [1]; `total` reusing `subtotal + self.TAX_RATE` [1].

Outline

- 1 Part A — Python foundations
- 2 Part B — Data, pandas, viz & statistics
- 3 Part C — ML, evaluation & feature engineering
- 4 Part D — AI engineering
- 5 Part E — Applied & production

Correct: B) `orders[(orders.returned == 1) & (orders.total > 100)]`.

- Combine boolean Series with `&` (element-wise), and **wrap each condition in parentheses** because `&` binds tighter than the comparison operators.

Why the others are wrong:

- A) Python `and` on a Series raises `ValueError` (ambiguous truth value).
- C) Without parentheses, `1 & orders.total` is evaluated first $\Rightarrow$ wrong result / error.
- D) A comma inside `[]` is not valid row-filter syntax.

Correct: B) HAVING.

- HAVING filters *groups* after GROUP BY + aggregation — it can see aggregates like SUM(total).

Why the others are wrong:

- A) WHERE filters *raw rows* before grouping and cannot reference aggregates.
- C) GROUP BY defines the groups; it does not filter them.
- D) ORDER BY only sorts the final result.

Logical execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.

```
orders.groupby("category")["returned"].mean().sort_values(ascending=False)
```

Why this works:

- returned is a 0/1 column, so its **mean is exactly the fraction of returned orders** (the return rate) — the workhorse trick for rates.
- .groupby("category") splits by category; .sort_values(ascending=False) ranks highest first.
- Marks: groupby+column [1]; .mean() as the rate [1]; descending sort [1].

```
from scipy import stats

result = stats.ttest_ind(express, standard, equal_var=False) # Welch's t-test
print(f"t = {result.statistic:.3f}, p = {result.pvalue:.4f}")
```

Notes:

- **Welch's two-sample t-test** (`equal_var=False`) is correct because variances and sample sizes are unequal — do *not* assume equal variance.
- Read the p-value from `result.pvalue`.
- Marks: correct function [2]; `equal_var=False` + naming Welch [1]; printing pvalue [1].

```
SELECT category, SUM(total) AS revenue
FROM orders
GROUP BY category
HAVING SUM(total) > 10000
ORDER BY revenue DESC;
```

Notes:

- GROUP BY category + SUM(total) aggregates revenue per category.
- The group filter must use HAVING (not WHERE).
- ORDER BY revenue DESC may reference the SELECT alias (ORDER BY runs after SELECT).
- Marks: GROUP BY + SUM [2]; HAVING filter [1]; ORDER BY DESC [1].

Bug: WHERE runs *before* grouping/aggregation, so SUM(total) does not yet exist there — the engine raises a “misuse of aggregate” error. Group filters belong in HAVING.

```
SELECT category, SUM(total) AS revenue
FROM orders
GROUP BY category
HAVING SUM(total) > 10000; -- was: WHERE SUM(total) > 10000
```

- Marks: identify that an aggregate cannot appear in WHERE [2]; correct HAVING rewrite [1].

Correct: B). At $\alpha = 0.05$ we have evidence the means differ, but the p-value alone says nothing about the *size* or importance of the effect.

- Always pair a p-value with an **effect size** (e.g. Cohen's d) and a **confidence interval**.

Why the others are wrong:

- A) A p-value is not the probability the means are equal.
- C) Significance $\neq$ large/important — with enough data a tiny difference becomes significant.
- D) “97% chance the result is real” is a common but false restatement of $1 - p$.

Outline

- 1 Part A — Python foundations
- 2 Part B — Data, pandas, viz & statistics
- 3 Part C — ML, evaluation & feature engineering**
- 4 Part D — AI engineering
- 5 Part E — Applied & production

Correct: B) tn, fp, fn, tp.

- `confusion_matrix` lays rows out as *actual* (negative, positive) and columns as *predicted* (negative, positive); flattening row-major gives [tn, fp, fn, tp].

Why the others are wrong: A, C, D all scramble that fixed order — unpacking into the wrong names silently corrupts every downstream precision/recall number.

Correct: B). Fitting the scaler on the whole dataset lets **test-set statistics (mean/std) leak into training**, so the reported score is optimistically inflated and won't hold in production.

- Fix: put the scaler *inside* a Pipeline so it is fitted on the training rows of each split/fold only.

Why the others are wrong:

- A) True that trees are scale-invariant, but that is not *why* pre-split scaling is wrong here.
- C) Scaling doesn't change class balance.
- D) StandardScaler can be re-fitted; that's not the issue.

Given **TP = 40, FP = 10, FN = 20, TN = 930**:

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{40}{40 + 10} = \frac{40}{50} = 0.80$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{40}{40 + 20} = \frac{40}{60} \approx 0.667$$

- Precision = “of the orders we *flagged*, how many were truly returned?”
- Recall = “of the orders that were truly returned, how many did we *catch*?”
- Marks: precision formula + value [1.5]; recall formula + value [1.5]. TN is not used.

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

pipe = Pipeline([("scaler", StandardScaler()),
                  ("clf", LogisticRegression(max_iter=1000))])
pipe.fit(X_tr, y_tr)
y_pred = pipe.predict(X_te)
```

Marks: split with `test_size=0.20 + stratify=y [1.5]`; `Pipeline(StandardScaler, LogisticRegression) [1.5]`; fit on train + predict on test [1]. The Pipeline is what makes it leakage-free.

(a) Break-even churn probability:

$$p^* = \frac{C}{s \cdot \text{CLV}} = \frac{8}{0.25 \times 400} = \frac{8}{100} = 0.08$$

(b) Expected value at $p = 0.10$:

$$EV = p \cdot s \cdot \text{CLV} - C = 0.10 \times 0.25 \times 400 - 8 = 10 - 8 = + \text{EUR } 2$$

(c) **Yes, send the offer:** the predicted churn probability $0.10 > p^* = 0.08$, equivalently the expected value is positive (+ EUR 2).

- Marks: p^* [2]; EV [2]; correct decision with justification [1]. Note the threshold is *per customer* — it scales with CLV.

Bug: the code divides by $(tp + fn)$ — that is **recall**. Precision divides by $(tp + fp)$.

```
tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()
precision = tp / (tp + fp) # was: tp / (tp + fn)
```

- Mnemonic: *precision* is about what you *predicted positive* (add the *false positives*); *recall* is about the *actual positives* (add the *false negatives*).
- Marks: recognise it computes recall / wrong denominator [2]; correct $tp/(tp+fp)$ [2].

Outline

- 1 Part A — Python foundations
- 2 Part B — Data, pandas, viz & statistics
- 3 Part C — ML, evaluation & feature engineering
- 4 Part D — AI engineering**
- 5 Part E — Applied & production

Correct: C) 0.0.

- temperature=0.0 makes the model (near-)deterministic — the right choice for classification, extraction and any task you want to be reproducible/testable.

Why the others are wrong:

- A) 1.0 and B) 0.7 inject randomness, appropriate for creative rewriting/brainstorming, not labelling.
- D) False — temperature very much affects output variability.

Correct: A) tools, resources, prompts.

- MCP's three primitives: **tools** (model-controlled actions), **resources** (application-controlled data, read by URI), **prompts** (user-controlled templates, like slash commands).

Why the others are wrong:

- B) host / client / server are the three *roles*, not the primitives (a tempting mix-up).
- C) GET/POST/PUT are HTTP methods; MCP rides on JSON-RPC 2.0.
- D) faithfulness/relevancy/recall are Ragas RAG-evaluation metrics.

(a) L2-normalize so that **vector magnitude / document length cannot fake similarity** — cosine should measure *direction* (meaning), not length; otherwise a longer document scores artificially high.

(b) For unit-length (L2-normalized) vectors, $|a| = |b| = 1$, so

$$\cos(a, b) = \frac{a \cdot b}{|a||b|} = a \cdot b$$

cosine similarity reduces to the plain **dot product**. That is why top- k retrieval over a normalized matrix collapses to a single matrix multiply (docs @ q).

- Marks: length/magnitude reasoning [2]; “dot product” [1].

```
import numpy as np

def cosine(a, b):
    return a @ b / (np.linalg.norm(a) * np.linalg.norm(b))

def top_k(q, docs, k):
    sims = np.array([cosine(q, d) for d in docs])
    return np.argsort(-sims)[:k] # indices of the k most similar docs
```

Notes:

- `a @ b` is the dot product; divide by the norms for cosine.
- `np.argsort(-sims)` sorts *descending*; slice `[:k]` for the top k .
- Vectorised alternative if docs rows are L2-normalized: `np.argsort(-(docs @ q))[:k]`.
- Marks: cosine [2]; correct top- k by descending similarity [2].

```
get_return_rate_tool = {
  "name": "get_return_rate",
  "description": "Return the return rate for a given product category.",
  "parameters": {
    "type": "object",
    "properties": {
      "category": {
        "type": "string",
        "description": "Product category, e.g. 'mugs'."
      }
    },
    "required": ["category"],
  },
}
```

Marks: name + description [1]; parameters as a JSON-Schema object with a string category property [1]; "required": ["category"] [1]. The description is what the model reads to decide when to call it.

Why it crashes: real LLM output is not guaranteed to be valid JSON — it may add markdown "‘ json fences or trailing prose — so `json.loads` raises `JSONDecodeError`.

Always wrap it in `try/except`.

```
def classify(review):  
    r = llm.chat(messages=[{"role": "user", "content": review}])  
    try:  
        return json.loads(r["text"])  
    except json.JSONDecodeError:  
        return {"sentiment": "unknown", "_raw": r["text"]}
```

- Marks: identify the unguarded parse / non-JSON output [1.5]; `try/except` with a structured fallback [1.5]. (Bonus-worthy: also set `temperature=0` and ask for “JSON only”.)

(a) The validator only checks the **shape** of the value: "ORD" is a non-empty string of the right type, so it *passes*. Validation cannot tell that it is a *truncated*, wrong value — it checks *shape*, *not truth*.

(b) To catch a value that is well-formed but wrong you need **ground truth** — a hand-labelled **golden set** to measure field-level accuracy against. Only comparing to known-correct answers reveals that "ORD" $\neq$ "ORD-5521".

- Marks: “shape not truth” explanation [1.5]; ground truth / golden set [1.5].

Outline

- 1 Part A — Python foundations
- 2 Part B — Data, pandas, viz & statistics
- 3 Part C — ML, evaluation & feature engineering
- 4 Part D — AI engineering
- 5 Part E — Applied & production

Correct: B). A shuffled split scatters days so **future dates land in training** — the model peeks at the future, test error looks great, then collapses on genuinely forward-only production data.

- Fix: split by time (`.iloc[:cut]` / `.iloc[cut:]`, or `TimeSeriesSplit`); every test point must come *after* every training point.

Why the others are wrong: A) speed is irrelevant; C) it's not about data volume; D) leakage affects both additive and multiplicative series.

Correct: B) copy requirements.txt and run pip install *before* copying the source.

```
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt
```

```
COPY src/ ./src/ # changes often -> only this layer rebuilds
```

- A code-only change invalidates only the late COPY src/ layer; pip install stays cached. Rule: least-likely-to-change instructions first.

Why the others are wrong: A) COPY . . first re-runs pip install on every code edit; C) python:latest is a non-reproducible moving tag; D) CMD order doesn't affect the install cache.

Actual [100, 200, 50], forecast [110, 180, 60]; absolute errors [10, 20, 10].

$$\text{MAE} = \frac{10 + 20 + 10}{3} = \frac{40}{3} \approx \mathbf{13.33}$$

$$\text{MAPE} = \frac{1}{3} \left(\frac{10}{100} + \frac{20}{200} + \frac{10}{50} \right) \times 100 = \frac{0.1 + 0.1 + 0.2}{3} \times 100 \approx \mathbf{13.33\%}$$

- MAE is in demand units; MAPE is scale-free (%). Marks: MAE [1.5]; MAPE [1.5]. (Recall MAPE explodes if any actual is 0.)

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans

vec = TfidfVectorizer(stop_words="english")
X = vec.fit_transform(reviews)

km = KMeans(n_clusters=3, n_init=10, random_state=42)
labels = km.fit_predict(X)
print(labels) # one cluster id per review
```

Marks: `TfidfVectorizer(stop_words="english") + fit_transform [2];`
`KMeans(n_clusters=3, n_init=10, random_state=...) [1]; fit_predict` producing
per-review labels [1]. (Top terms per topic =
`np.argsort(km.cluster_centers_[c])[:, :-1][:n].)`

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(html, "html.parser")
products = []
for li in soup.find_all("li", class_="product"):
    name = li.find("h2", class_="name").get_text(strip=True)
    price = li.find("span", class_="price").get_text(strip=True)
    products.append((name, price))
```

Marks: BeautifulSoup(html, "html.parser") [1]; find_all("li", class_="product") [1]; extract name + price via find(...).get_text() (or .text) [1]; build (name, price) tuples [1]. (See E6 for the robust version.)

Why it crashes: when `find(...)` matches nothing it returns `None`, and `None.text` raises `AttributeError`. Capture the node, test for `None`, *then* read it.

```
node = li.find("h2", class_="name")
name = node.get_text(strip=True) if node is not None else None
```

- A resilient extractor returns an all-None row instead of crashing, so you can *alert on empty rows* rather than get paged at 3 a.m.
- Marks: explain `find` returns `None` [1.5]; None-guarded rewrite [1.5].

Essential step: standardize the RFM features (e.g. `StandardScaler`) before clustering.

- KMeans uses Euclidean distance. **Monetary** has by far the largest magnitude, so without scaling it *dominates the distance* and the clustering degenerates into “sort customers by spend” — Recency and Frequency are effectively ignored.
- Scaling puts all three axes on comparable footing so segments reflect the full RFM shape (champions, at-risk big spenders, bargain newcomers, ...).
- Marks: name scaling/standardization [1]; explain the largest-magnitude-feature-dominates failure [1].

Mark map — 100 total

- **Part A** $2+2+3+4+4+5 = 20$ **Part B** $2+2+3+4+4+3+2 = 20$
 - **Part C** $2+2+3+4+5+4 = 20$ **Part D** $2+2+3+4+3+3+3 = 20$
 - **Part E** $2+2+3+4+4+3+2 = 20$
-
- Recurring themes: rates as means of 0/1 columns; leakage = information from the future / the answer; validation checks *shape*, ground truth checks *truth*; decisions need a *costed threshold*, not just a probability.
 - This is one of three parallel-form mock exams — sit all three for full coverage.